

1. Il problema: perché la recall è bassa?

I sistemi di IR classici (VSM, BM25, Language Models) si basano sul **term matching**: un documento viene recuperato solo se contiene esattamente i termini della query.

Questo approccio fallisce in due scenari tipici:

Sinonimia — l'utente scrive `aircraft`, ma i documenti rilevanti usano `plane`, `airplane`, `aviation`. Non c'è overlap lessicale, quindi i documenti non vengono recuperati, pur essendo rilevanti.

Vocabolario divergente — l'utente usa parole comuni (`heart attack`), ma la letteratura specialistica usa termini tecnici (`myocardial infarction`).

Ricordiamo la definizione di **recall**:

Recall = TP / (TP + FN) = (rilevanti recuperati) / (rilevanti totali)

I documenti rilevanti non recuperati (falsi negativi, *FN*) abbassano la recall. L'obiettivo delle tecniche che seguono è ridurre *FN* espandendo la capacità del sistema di trovare documenti rilevanti al di là dei termini esatti.

Nota: recall e precision sono spesso in conflitto. Recuperare più documenti aumenta la recall ma rischia di abbassare la precision se si includono documenti non rilevanti. Tutte le tecniche discusse qui devono tenere conto di questo trade-off.

2. Query Expansion

L'idea fondamentale è **espandere la query originale aggiungendo termini correlati**, così da recuperare documenti che non contengono le parole esatte dell'utente ma che trattano lo stesso argomento.

Esistono due famiglie di approcci, distinte in base a quando e come viene costruita la risorsa di espansione.

2.1 Metodo Locale — Relevance Feedback

Il sistema usa il feedback dell'utente **sulla singola query corrente** per riformulare la query stessa. È detto *locale* perché la risorsa di espansione non esiste a priori: viene costruita al volo a partire dai risultati restituiti.

Procedura:

- 1. L'utente formula una query iniziale q_0 .
- 2. Il sistema restituisce una prima lista di risultati.
- 3. L'utente marca alcuni documenti come **rilevanti** (D_r) e altri come **non rilevanti** (D_{nr}).
- 4. Il sistema sfrutta questi giudizi per costruire una query modificata q_m che si avvicina ai documenti rilevanti e si allontana da quelli non rilevanti.
- 5. Il processo può essere **iterato** più volte.

Fondamento teorico: query ottimale e centroide

Nel VSM ogni documento è un vettore nello spazio dei termini. Il **centroide** di un insieme di documenti D è il loro punto medio:

μ(D) = 1/|D| ∑ d

Se conoscessimo perfettamente l'insieme di tutti i documenti rilevanti C_r e non rilevanti C_{nr} , la query ottimale sarebbe:

q_opt = μ(C_r) - μ(C_nr)

ovvero il vettore che minimizza la distanza dal centroide dei rilevanti e la massimizza da quello dei non rilevanti. Naturalmente, conoscere C_r e C_{nr} a priori risolverebbe già il problema: per questo si approssima tramite l'algoritmo di **Rocchio**.

Algoritmo di Rocchio

Rocchio è l'implementazione classica del relevance feedback per VSM. Partendo dalla query originale q_0 , la nuova query modificata è:

q_m = αq_0 - β(1/|D_r| ∑ d) + γ(1/|D_nr| ∑ d)

Simbolo	Ruolo
α	peso della query originale (fedeltà all'intento iniziale)
β	peso del centroide dei rilevanti (attrazione verso argomenti utili)
γ	peso del centroide dei non rilevanti (repulsione da argomenti indesiderati)

Tipicamente $\alpha > \beta > \gamma \geq 0$, e γ è spesso posto a zero in pratica per evitare che la sottrazione produca pesi negativi ingestibili.

Assunzioni necessarie

Rocchio funziona bene solo se valgono due condizioni:

A1 — Vocabolario condiviso: l'utente deve usare parole abbastanza vicine a quelle della collezione, altrimenti già la prima query non recupera nulla su cui iterare. Se l'utente cerca `cosmonaut` ma la collezione usa `astronaut`, il punto di partenza è troppo lontano.

A2 — Omogeneità dei rilevanti: i documenti rilevanti devono condividere termini simili. Se una query come *"personaggio con doppia vita"* ha come rilevanti documenti su Batman, su Dr. Jekyll e su spie, i loro centroidi si trovano in regioni molto distanti dello spazio, e il feedback su uno peggiora il recupero degli altri.

Pseudo-Relevance Feedback

Per evitare di chiedere all'utente di esprimere giudizi espliciti, si usa il **Pseudo-Relevance Feedback (PRF)**:

1. Si recuperano i primi k documenti.
2. Si **assume automaticamente** che siano rilevanti.
3. Si applica Rocchio come se fossero stati marcati dall'utente.

Il vantaggio è l'assenza di interazione: il sistema si auto-migliora senza chiedere nulla all'utente. Il rischio è il **query drift**: se i primi k documenti non sono effettivamente rilevanti, la query si sposta verso argomenti sbagliati. Dopo una o più iterazioni, la query può derivare completamente dall'intento originale dell'utente.

2.2 Metodo Globale — Global Query Expansion

Invece di usare il feedback sulla singola query, si costruisce una risorsa **indipendente dalla query** che vale per tutta la collezione. Si parla di espansione *globale* perché avviene una sola volta per tutte.

Manual Thesaurus

Un **thesaurus manuale** è costruito da esperti del dominio e contiene relazioni esplicite tra termini: sinonimi, iperonimi, iponimi, termini correlati.

Esempio: in PubMed, gli esperti medici hanno costruito il thesaurus MeSH che collega `heart attack` a `myocardial infarction`.

Vantaggi: alta qualità, pochissime associazioni errate.

Svantaggi: costo di costruzione e manutenzione elevato; non si scala a domini generici.

Come si usa: per ogni termine della query si consultano le voci del thesaurus e si aggiungono i termini correlati con un peso inferiore a quello dei termini originali.

Automatically Derived Thesaurus

Si genera automaticamente a partire dalla collezione, osservando la **distribuzione dei termini nei documenti**: due parole sono simili se co-occorrono in contesti simili.

Due definizioni di similarità tra termini:

Definizione 1 — Co-occorrenza di parole: due parole sono simili se co-occorrono con parole simili. `car` $\approx$ `motorcycle` perché entrambi compaiono frequentemente vicino a `drive`, `engine`, `road`.

Definizione 2 — Relazioni grammaticali: due parole sono simili se entrano nelle stesse relazioni sintattiche. `apple` $\approx$ `orange` perché si può costruire `eat apples` / `eat oranges`, `peel apples` / `peel oranges`. Questa definizione è più precisa ma più costosa da calcolare.

Query Logs

La fonte principale nei motori di ricerca reali sono i **query log**: dati che registrano le query degli utenti, i documenti cliccati, e le riformulazioni della ricerca.

Due pattern utili:

- **Query successive:** chi cerca `herbs` spesso poi cerca `herbal remedies` $\rightarrow$ `herbal remedies` è una buona espansione di `herbs`.
- **Query diverse con lo stesso click:** chi cerca `batman` e chi cerca `bruce wayne` cliccano spesso sugli stessi documenti $\rightarrow$ le due query sono semanticamente equivalenti.

Differenza chiave rispetto ai thesauri: i thesauri si basano sui documenti e sulla semantica dei termini; i query log si basano sul **comportamento degli utenti**, catturando equivalenze che un thesaurus non conoscerebbe.

2.3 Pesi nell'espansione

Un tema trasversale a tutti i metodi di espansione è la gestione dei **pesi**: aggiungere nuovi termini con lo stesso peso dei termini originali rischia di spostare troppo la query e peggiorare la precision.

La formula generale è:

$$q' = \alpha \cdot q + \beta \cdot e$$

dove q è il vettore della query originale, e è il vettore dei termini aggiunti, e $\beta < \alpha$ garantisce che l'intento originale dell'utente rimanga dominante.

I pesi possono essere scelti in base alla fonte:

- **Thesaurus manuale:** pesi più alti per sinonimi stretti, più bassi per iperonimi e termini correlati.
- **Thesaurus automatico:** il peso è la misura di similarità tra il termine originale e quello espanso.
- **Query log:** il peso è la frequenza con cui il comportamento osservato si è verificato (più utenti hanno fatto quella riformulazione → peso maggiore).

3. Latent Semantic Indexing (LSI) e SVD

3.1 Il problema della semantica nel VSM

I modelli classici confrontano query e documenti nello **spazio originale dei termini**: ogni dimensione è una parola distinta. Questo approccio soffre di due problemi fondamentali:

Sinonimia: `car` e `automobile` sono termini diversi → vivono su assi diversi nello spazio → un documento con `automobile` e una query con `car` hanno bassa similarità anche se trattano lo stesso argomento.

Polisemia: `Saturn` può riferirsi al pianeta o all'azienda automobilistica. Il VSM non distingue i due contesti: un unico asse rappresenta entrambi i significati, portando a recuperare documenti di argomenti completamente diversi.

LSI affronta questi problemi costruendo uno **spazio latente** in cui termini e documenti semanticamente correlati finiscono vicini, indipendentemente dal fatto che condividano le stesse parole esatte.

3.2 Fondamenti algebrici: SVD

LSI si basa sulla **Singular Value Decomposition (SVD)**, una tecnica di algebra lineare applicabile a qualsiasi matrice rettangolare.

Matrice termine-documento

Il punto di partenza è la matrice $A \in \mathbb{R}^{m \times n}$, dove:

- le **righe** sono i termini (m termini nel vocabolario),
- le **colonne** sono i documenti (n documenti nella collezione),
- la cella $A_{t,d}$ contiene il peso TF-IDF del termine t nel documento d .

Decomposizione SVD

La SVD fattorizza A come prodotto di tre matrici:

$$A = U \Sigma V^T$$

Matrice	Dimensioni	Contenuto
U	$m \times m$	colonne = autovettori di AA^T (relazioni termine-termine)
Σ	$m \times n$	valori singolari $\sigma_i = \sqrt{\lambda_i}$ sulla diagonale, decrescenti
V^T	$n \times n$	righe = autovettori di $A^T A$ (relazioni documento-documento)

Interpretazione intuitiva:

- AA^T misura la relazione tra termini: due termini hanno un valore alto se co-occorrono negli stessi documenti. Gli autovettori di AA^T (colonne di U) sono le **direzioni latenti nello spazio dei termini**.
- $A^T A$ misura la relazione tra documenti: due documenti sono simili se contengono pattern di termini simili. Gli autovettori di $A^T A$ (colonne di V) sono le **direzioni latenti nello spazio dei documenti**.
- I **valori singolari** σ_i indicano l'importanza di ogni direzione latente: più è grande, più quella direzione cattura struttura rilevante nei dati.

3.3 Low-Rank Approximation

La matrice A originale è spesso sparsa, ad alta dimensionalità e rumorosa. L'idea di LSI è **sostituirla con una versione compressa di rango k** che conservi la struttura semantica principale eliminando il rumore.

Formalmente si cerca:

$$A_k = \arg \min_{\text{rank}(X)=k} \|A - X\|_F$$

dove $\|\cdot\|_F$ è la norma di Frobenius: $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$.

Il **teorema di Eckart-Young** garantisce che la soluzione ottimale si ottiene mantenendo solo i primi k valori singolari e azzerando gli altri:

$$A_k = U_k \Sigma_k V_k^T = \sum_{i=1}^k \sigma_i u_i v_i^T$$

dove ogni termine $\sigma_i u_i v_i^T$ è una matrice di rango 1 che rappresenta una **direzione latente** con importanza σ_i .

L'energia spiegata dalle prime k componenti è:

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_i \sigma_i^2}$$

Questa misura guida la scelta di k : si seleziona il valore minimo che spieghi una quota sufficiente dell'energia totale.

3.4 Rappresentazione nello spazio latente

Una volta calcolata la SVD troncata, termini e documenti vengono rappresentati nello spazio latente di k dimensioni.

Documenti:

$$D_k = \Sigma_k V_k^T \in \mathbb{R}^{k \times n}$$

Ogni colonna di D_k è il vettore latente di un documento. Moltiplicare per Σ_k scala le dimensioni in base alla loro importanza.

Termini:

$$T_k = U_k \Sigma_k \in \mathbb{R}^{m \times k}$$

Ogni riga di T_k è il vettore latente di un termine.

Rappresentazione simmetrica: per ricostruire la cella $A_k(t, d)$ come prodotto scalare diretto tra termine e documento, si usa la rappresentazione simmetrica con $\Sigma_k^{1/2}$:

$$T_k = U_k \Sigma_k^{1/2}, \quad D_k = \Sigma_k^{1/2} V_k^T$$

in modo che $A_k(t, d) \approx \langle T_k(t), D_k(d) \rangle$.

3.5 Folding-in: proiettare la query nello spazio latente

Una query $q \in \mathbb{R}^m$ nello spazio originale viene proiettata nello spazio latente con l'operazione detta **folding-in**:

$$q_k = \Sigma_k^{-1} U_k^T q$$

Perché questa formula? U_k proietta i termini verso le direzioni latenti; Σ_k^{-1} riscalda correttamente in base all'importanza di ogni dimensione. Il risultato è un vettore denso $q_k \in \mathbb{R}^k$: anche se la query originale era sparsa (poche parole), la sua proiezione latente attiva molte dimensioni, incluse quelle associate a sinonimi e concetti correlati.

Esempio: una query `laptop` dopo il folding-in può attivare le dimensioni latenti legate a `computer`, `display`, `software`, permettendo di recuperare documenti che non contengono la parola `laptop`.

Il ranking finale si ottiene calcolando la cosine similarity tra q_k e tutti i vettori in D_k .

3.6 LSI e il miglioramento della recall

LSI migliora la recall per due meccanismi:

Sinonimia: due termini come `car` e `automobile` compaiono in documenti simili → nella decomposizione SVD i loro vettori latenti finiscono vicini → una query con `car` può recuperare documenti con `automobile` e viceversa.

Polisemia parziale: un termine polisemico come `Saturn` viene proiettato verso tutte le dimensioni latenti dei suoi diversi significati. Nota: LSI **non crea due vettori separati** per i diversi significati di una stessa parola — questa rimane una limitazione del modello.

Evidenza empirica: negli esperimenti originali degli anni '90, LSI con $k \in [250, 350]$ dimensioni raggiungeva precision superiore alla mediana del benchmark TREC, migliorando la recall rispetto al VSM classico.

3.7 LSI per Query Expansion

Lo spazio latente di LSI può essere usato direttamente per la query expansion: si cercano i termini più vicini alla query nello spazio latente e li si aggiunge alla query originale, rispettando il principio dei pesi differenziati visto in § 2.3 ($\beta < \alpha$).

Strategia a due fasi (candidate generation + re-ranking):

Applicare LSI all'intera collezione è costoso ($O(n)$ confronti). In pratica si usa una strategia ibrida:

- Candidate generation:** si usa l'indice inverso tradizionale (BM25 o simile) per recuperare un insieme ristretto di N documenti candidati, eventualmente con query expansion basata su LSI.
- Re-ranking:** si applica LSI solo ai candidati per calcolare la similarità nello spazio latente e produrre il ranking finale.

Questo abbina l'efficienza dell'indice inverso alla qualità semantica di LSI.

3.8 Limiti di LSI

Nonostante i vantaggi, LSI presenta alcune limitazioni importanti:

Negazioni e query booleane: LSI non gestisce vincoli logici come `jaguar NOT car`. Lavorando con similarità vettoriale e associazioni semantiche, tratta tutti i termini come attrattori positivi verso regioni dello spazio, ignorando la semantica delle negazioni.

Precision percepita: migliorare la recall significa anche rischiare di recuperare documenti semanticamente correlati ma non rilevanti per la specifica query. Nel caso estremo, una query su `pompa` in gergo petrolifero può restituire come primo risultato `albero di Natale` (un tipo di pompa nel settore), disorientando l'utente.

Scalabilità: la SVD su grandi collezioni è computazionalmente costosa. Si ovvia con algoritmi iterativi che calcolano solo i primi k vettori singolari, e con la strategia a due fasi descritta sopra.

Non è un modello neurale: la semantica catturata da LSI deriva dalla struttura globale delle co-occorrenze nella collezione, non da un addestramento predittivo su un task. LSI non generalizza come BERT o i modelli transformer.

4. Confronto tra le tecniche

	Relevance Feedback	Global QE	LSI
Risorsa usata	Giudizi dell'utente	Thesaurus / Query log	Struttura statistica della collezione
Costruzione	Online (per ogni query)	Offline (una volta)	Offline (una volta)
Migliora la recall	✓	✓	✓
Rischio query drift	Alto (spec. PRF)	Moderato	Basso (pesi automatici)
Richiede interazione utente	Sì (RF) / No (PRF)	No	No
Gestisce sinonimia	Indirettamente	Dipende dal thesaurus	✓ (direttamente)
Gestisce polisemia	No	No	Parzialmente
Costo computazionale	Basso	Basso	Alto (SVD)

5. Osservazioni conclusive

Le tre famiglie di tecniche affrontano il problema della recall da prospettive complementari. Il **Relevance Feedback** sfrutta il feedback esplicito dell'utente per muovere la query verso la regione giusta dello spazio vettoriale. La **Query Expansion globale** costruisce a priori una mappa di relazioni tra termini e la usa per arricchire qualsiasi query. **LSI** ridefinisce lo spazio di rappresentazione stesso, portando termini e documenti semanticamente correlati in prossimità anche senza overlap lessicale esatto.

In un sistema reale moderno queste tecniche non sono mutuamente esclusive: BM25 o un indice inverso gestisce la candidate generation, LSI o un modello neurale si occupa del re-ranking semantico, e la query expansion può intervenire in entrambe le fasi per aumentare il bacino di candidati rilevanti.